SOFTWARE REQUIREMENTS SPECIFICATION
DockWatch: Docker Container Health Monitoring System
Version: 1.0 | **Date:** April 15, 2026
Prepared For: Software Developers, QA Engineers, System Administrators, Academic Evaluators

---

1. INTRODUCTION

1.1 Purpose
The purpose of this document is to define the functional and non-functional requirements for **DockWatch**, a centralized Docker container health monitoring system. DockWatch is designed to:
- Reduce system downtime through automated failure detection and recovery mechanisms.
- Provide historical performance data to support infrastructure capacity planning.
- Centralize container monitoring into a single, unified interface.
- Enable proactive infrastructure management across development and production environments.

Key Performance Objectives:
- Monitor up to **50 concurrent containers** with **<5% host CPU overhead**.
- Detect container failures within **30 seconds** of occurrence.
- Achieve **99.9% system uptime** for the monitoring service itself.
- Deliver **sub-200ms API response times** at the 95th percentile.

1.2 Scope
DockWatch will operate as a lightweight, standalone monitoring agent/service that interfaces directly with the Docker Engine. It will collect real-time telemetry, expose REST and WebSocket APIs for integration, and provide a web-based UI/UX dashboard. The system will support configuration via YAML, secure communication via TLS/JWT, and data serialization using JSON. It will not modify container runtime behavior but will observe, alert, and trigger predefined recovery scripts via standard Docker APIs.

1.3 Definitions, Acronyms, and Abbreviations
Term	Definition
API	Application Programming Interface
CI/CD	Continuous Integration/Continuous Deployment
CPU	Central Processing Unit
DFD	Data Flow Diagram
Docker	Platform for developing, shipping, and running containerized applications
HTTP	Hypertext Transfer Protocol
I/O	Input/Output
IEEE	Institute of Electrical and Electronics Engineers
JSON	JavaScript Object Notation
JWT	JSON Web Token
PID	Process Identifier

RAM	Random Access Memory
REST	Representational State Transfer
SRS	Software Requirements Specification
TLS	Transport Layer Security
TTL	Time To Live
UI	User Interface
UX	User Experience
WebSocket	Protocol providing full-duplex communication over TCP
YAML	YAML Ain't Markup Language (human-readable data serialization)

1.4 References
1. IEEE Std 830-1998, *IEEE Recommended Practice for Software Requirements Specifications*
2. Docker Engine API v1.41 Documentation (`https://docs.docker.com/engine/api/`)
3. OWASP Top 10 Security Risks (`https://owasp.org/www-project-top-ten/`)
4. ISO/IEC 25010:2011, *Systems and software Quality Requirements and Evaluation (SQuaRE)*

1.5 Document Overview
This SRS is structured as follows:
- **Section 1:** Introduction and project context
- **Section 2:** Overall system description, constraints, and user perspectives
- **Section 3:** Specific detailed requirements (functional and non-functional)
- **Section 4:** Appendices including use cases and requirements traceability matrices

---

2. OVERALL DESCRIPTION

2.1 Product Perspective
DockWatch operates as a complementary monitoring layer within existing Docker-based infrastructure. It communicates with the Docker Engine API v1.41 to poll container states, resource utilization (CPU, RAM, I/O), and health check statuses. The system exposes a RESTful API for external integrations and a WebSocket endpoint for real-time dashboard updates. It is designed to integrate seamlessly into CI/CD pipelines and existing observability stacks without requiring container image modifications.

2.2 User Characteristics
- **Software Developers & Architects:** Utilize historical performance data and API access to optimize container configurations and validate architectural decisions.
- **Quality Assurance Engineers:** Leverage monitoring logs and failure timelines to reproduce edge cases and validate system resilience during testing phases.
- **System Administrators:** Primary operators who configure alerts, manage recovery policies, and monitor the centralized UI/UX for real-time infrastructure health.
- **Academic Evaluators:** Reference the system’s architecture, compliance standards (ISO/IEC 25010), and performance metrics for educational or research purposes.

2.3 Operating Environment & Constraints

- **Runtime:** Linux-based hosts running Docker Engine (API v1.41+)
- **Network:** Requires HTTP/HTTPS and WebSocket connectivity; all external traffic must be secured via TLS.
- **Security:** Authentication via JWT; strict adherence to OWASP Top 10 mitigation practices.
- **Resource Constraints:** Must maintain <5% CPU overhead on the host when monitoring up to 50 concurrent containers.
- **Reliability:** The monitoring service itself must achieve 99.9% uptime, independent of monitored container states.

2.4 Assumptions & Dependencies
- Docker daemon is running and API socket is accessible.
- Host system maintains stable network connectivity for alerting and remote API access.
- Configuration files are provided in valid YAML format.
- Telemetry data retention policies are managed externally or via configurable TTL parameters.

---

3. SPECIFIC REQUIREMENTS

3.1 Functional Requirements
ID	Requirement	Priority
FR-01	System shall continuously poll Docker Engine API to discover running containers and track PID, CPU, and RAM usage.	High
FR-02	System shall detect container failures, unresponsive states, or health check violations within 30 seconds.	High
FR-03	System shall automatically trigger predefined recovery actions (restart, isolate, or notify) based on configurable rules.	High
FR-04	System shall store historical performance metrics and exposure data via REST API for capacity planning.	Medium
FR-05	System shall provide a centralized UI/UX dashboard displaying real-time status, alerts, and historical trends.	High
FR-06	System shall support configuration management via YAML files and export telemetry in JSON format.	Medium

3.2 Non-Functional Requirements
ID	Category	Requirement	Target/Metric
NFR-01	Performance	API response time	≤ 200ms at 95th percentile
NFR-02	Resource Utilization	Host CPU overhead	< 5% when monitoring 50 containers
NFR-03	Availability	Monitoring service uptime	99.9%
NFR-04	Security	Authentication & Transport	JWT for auth; TLS 1.2+ for all HTTP/WebSocket traffic
NFR-05	Scalability	Concurrent container monitoring	Up to 50 containers per agent instance

| NFR-06 | Maintainability | Compliance & Documentation | Aligns with ISO/IEC 25010 quality model; structured per IEEE 830-1998 |
| NFR-07 | Interoperability | API & Data Formats | RESTful endpoints, JSON payloads, YAML configuration, WebSocket streaming |

3.3 External Interface Requirements
- **User Interface:** Web-based dashboard optimized for UX, featuring responsive layouts, real-time alert panels, and historical graphing.
- **Hardware Interfaces:** Standard x86_64/ARM64 hosts with ≥2 GB RAM and 2 vCPUs recommended for optimal agent performance.
- **Software Interfaces:** Docker Engine API v1.41, standard HTTP/HTTPS protocols, JSON/YAML parsers.
- **Communication Interfaces:** Full-duplex WebSocket for live updates; REST for batch/query operations; TTL-based cache for transient states.

---

4. APPENDICES

Appendix A: Use Case Summary
1. **UC-01:** Admin configures monitoring thresholds via YAML → System validates and applies rules → Dashboard reflects new policies.
2. **UC-02:** Container crashes → DockWatch detects failure within 30s → Triggers restart & logs event → Sends alert via API/WebSocket.
3. **UC-03:** QA engineer queries historical CPU/RAM data via REST API → Receives JSON payload → Uses data for performance regression testing.

Appendix B: Requirements Traceability Matrix
Requirement	Stakeholder	Design Phase	Test Method
FR-01, NFR-02	Sys Admins, Devs	Architecture	Load Testing
FR-02, FR-03	QA Engineers, Sys Admins	Failure Simulation	Chaos Engineering
FR-04, FR-05	Academic Evaluators, QA	Data Layer	API Validation
NFR-03, NFR-04	All	Security & HA	Penetration & Uptime Monitoring

---
Document Approval
Prepared By: Systems Engineering Team | *Reviewed By:* QA & Security Leads | *Approved By:* Project Sponsor
Status: Baseline v1.0 | *Next Review:* Q3 2026